湖南科技大学计算机科学与工程学院

计算机组成原理 课程设计报告

专业班级： 24 级物联网工程三班

姓　　名： 李思民

学　　号： 2405040303

指导教师： 张会福

时　　间： 2025-2026-1 学期第 18 周

地　　点： 逸夫楼 535

指导教师评语：

成绩：　　**等级：**

签名：

年　　月　　日

实验 0 ROM 仿真

一、实验名称

预备实验 ROM 仿真

二、实验目的

1．深入理解只读存储器（ROM）的核心工作原理，掌握存储元由二极管构成的逻辑机制，明确 “有二极管为 1、无二极管为 0” 的存储规则；

2．具备独立设计 ROM 仿真电路的能力，能够绘制规范、整齐的逻辑电路图和接线图，确保电路布局合理、信号流向清晰；

3．熟练掌握 ROM 的数据读出原理及操作流程，能够通过地址选择实现指定存储单元的数据读取；

4．学会将自定义数据（本人学号后 4 位）写入 ROM 并完成验证，确保数据存储与读出的准确性，提升实践操作与数据验证能力。

三、总体设计

1.基本原理

ROM 作为非易失性存储器，其核心特征是断电后存储数据不丢失，本实验采用 4 字 ×4 位的结构设计，即包含 4 个存储单元，每个单元存储 4 位二进制数据。存储元的核心构成是二极管，其工作逻辑为：当某一存储单元的对应位接入二极管时，二极管正向导通，使该位数据线获得高电平，对应输出逻辑 “1”；当未接入二极管时，数据线通过限流电阻接地，输出低电平，对应逻辑 “0”。

实验中通过 2 位地址信号（由两个按键控制）实现 4 个存储单元（地址编码 00、01、10、11）的选通，地址信号经反相器和与门逻辑处理后，精准选中目标存储单元，该单元存储的 4 位二进制数据通过数据线传输至数码管，最终以十六进制格式直观显示，完成数据读出过程。

2.模块介绍

(1) 存储模块：由二极管阵列、与门（7408N）、非门（7404N）构成，实现 4 个 4 位数据的存储，地址由按键控制；

(2) 控制模块：按键（Key-Space）作为地址选择信号，控制不同存储单元的选通；

(3) 显示模块：DCD_HEX 数码管，将 ROM 输出的 4 位二进制数据转换为十六进制显示，直观呈现读出结果；

(4) 电源模块：5V 直流电源（VCC）为整个电路提供工作电压，1kΩ 电阻用于限流保护。

四、详细设计

1.设计步骤

(1) 确定存储数据：提取学号后 4 位（学号 0303），转换为 4 个 4 位二进制数（0→0000、3→0011、0→0000、3→0011），作为 ROM 的 4 字存储内容；

(2) 元件选型与准备：根据实验原理选取 7408N（与门）、7404N（非门）、1N4001 二极管、按键、1kΩ电阻、DCD_HEX 数码管、VCC 电源及接地元件；

绘制逻辑电路图：按 “地址输入→存储阵列→数据输出→显示” 的信号流向，布局元件并完成接线；

2.电路结构

本实验的 ROM 仿真电路结构核心是通过地址选通存储单元，实现数据的存储与读出，ROM 电路的设计如图 0.1 所示，现对电路结构进行介绍：

基于 Multisim 搭建的 ROM 仿真电路整体布局，核心模块包括地址控制模块（2 个按键）、地址处理模块（7404N 非门、7408N 与门）、存储模块（1N4001 二极管阵列）、限流保护模块（4 个 1kΩ 电阻）和显示模块（DCD_HEX 数码管）。电源模块提供 5V 工作电压，各模块按 “地址输入→处理→存储→输出→显示” 的信号流向接线，确保数据传输路径清晰。

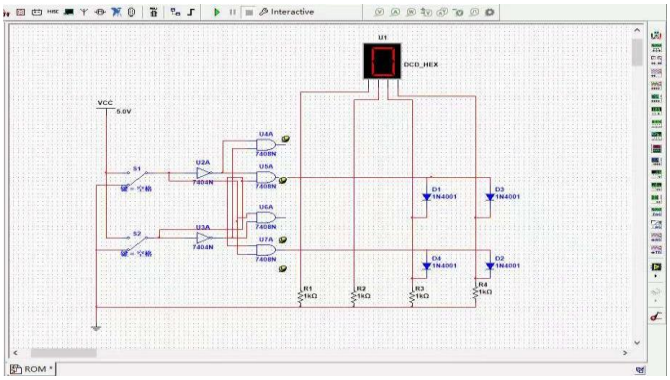


图 0.1 ROM 电路表

(1) 电源端接入 5V VCC，为 7404N、7408N、存储阵列及 DCD_HEX 数码管提供工作电压等芯片及存储阵列提供工作电压；接地元件为整个电路提供公共地，确保电路电平参考一致。

(2) 两个按键（S1、S2）作为地址输入设备，按键未按压时输出高电平，按压时输出低电平。按键的输出端分别连接至 7404N 非门的两个输入端，经非门反相处理后，输出高电平和低电平信号，形成 2 位地址信号 A1（对应 S1）和 A0（对应 S2），进而组合成 4 个地址编码（00、01、00、01），对应 4 个存储单元。7408N 与门芯片接收地址信号，通过逻辑与运算选通对应的存储支路，每个与门输出端连接一个存储支路。

(3) 存储支路由 1 个二极管和 1 个 1kΩ 限流电阻组成，二极管阳极接与门输出，阴极接数据线（对应数码管输入），电阻一端接地，用于稳定电平。

(4) 4 条数据线分别连接 DCD_HEX 数码管的 4 个输入引脚，实时显示读出的 4 位二进制数据（以十六进制形式呈现）。

3.元件功能与接线说明

(1) 7404N：六反相器芯片，本实验使用 2 个反相器，将按键输入的地址信号反相，确保地址逻辑的正确性。接线方式：按键输出端接反相器输入端，反相器输出端接 7408N 与门输入端。

(2) 7408N：四 2 输入与门芯片，使用 4 个与门，每个与门接收 2 位地址信号的不同组合，实现存储单元的选通。接线方式：与门两个输入端分别接地址信号或其反相信号，与门输出端接二极管阳极。

(3) 1N4001 二极管：作为存储元核心，接入对应支路表示存储 “1”，不接入表示存储 “0”。接线方式：阳极接与门输出，阴极接数码管对应输入引脚。

(4) 1kΩ电阻：限流电阻，防止电路电流过大损坏元件，同时稳定数据线电平。接线方式：一端接数据线，一端接地。

(5) DCD_HEX 数码管：用于显示读出的 4 位二进制数据，以十六进制格式输出，直观呈现实验结果。

五、实验结果与分析

1.地址选择功能验证：

通过按压不同组合的按键，成功实现了 4 个存储单元的选通，说明地址选择模块的逻辑设计正确，7404N 反相器和 7408N 与门的组合工作正常。

2.数据读出功能验证：

每个地址对应的读出结果与预设的学号后 4 位数据完全一致，实验结果如图 0.2、图 0.3、图 0.4、图 0.5 所示，实现了 0303 学号的存储，证明 ROM 的存储机制仿真成功，二极管作为存储元的逻辑功能符合预期

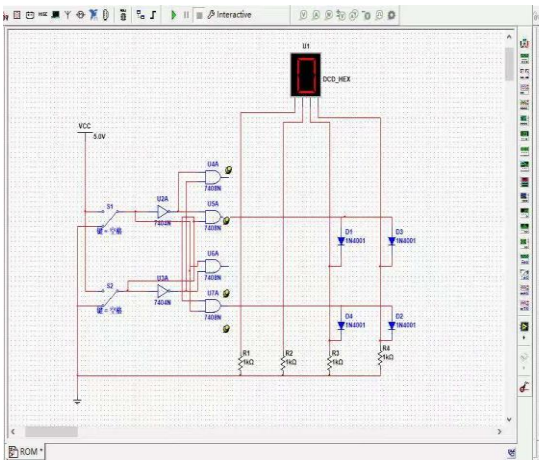


图 0.2 ROM 电路表

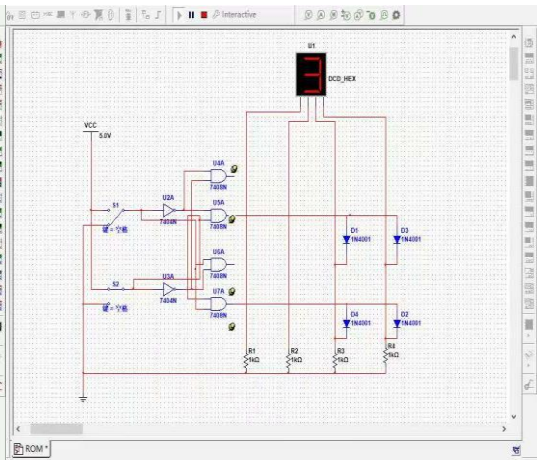


图 0.3 ROM 电路表

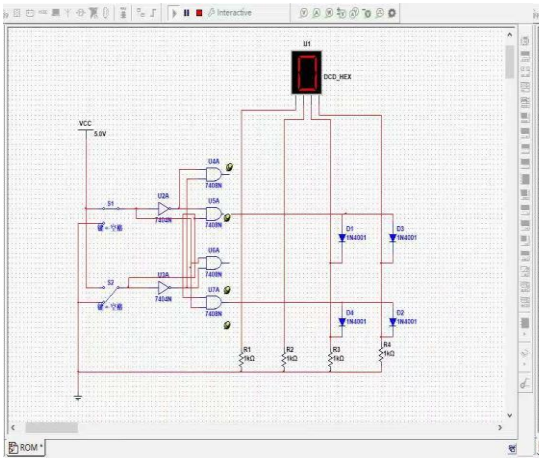


图 0.4 ROM 电路表

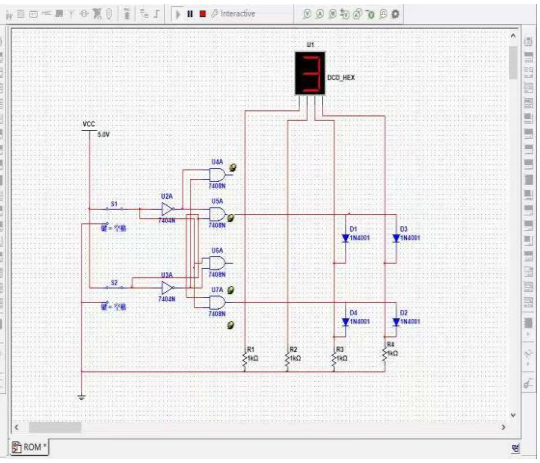


图 0.5 ROM 电路表

六、小结与心得体会

本次 ROM 仿真实验让我深入理解了只读存储器的工作本质，从理论上掌握了存储元的构成原理，在实践中学会了在 Multisim 平台搭建组合逻辑存储电路。实验中遇到的主要问题是初期接线时二极管的方向接反，导致数据读出错误，通过查阅元件手册和反复调试，最终解决了该问题。通过本次实验，我不仅熟练掌握了 7404N、7408N 等常用逻辑芯片的功能和使用方法，还提升了电路设计、接线和故障排查的能力。后续可进一步扩展实验，增加 ROM 的存储容量，或尝试设计可改写的 PROM 仿真电路。

实验 1 验证 74LS181 运算和逻辑功能

一、实验名称

验证 74LS181 运算和逻辑功能

二、实验目的

1. 深入理解算术逻辑单元（ALU）的核心工作原理，明确算术运算与逻辑运算的实现机制，掌握控制信号对运算类型和结果的影响规律；

2. 熟悉简单运算器的数据传送通路，理解运算数输入、控制信号传输、运算结果输出的完整流程，明确各模块间的协同工作关系；

3. 具备独立绘制 74LS181 验证电路逻辑电路图和规范接线图的能力，确保电路布局合理、信号流向清晰、接线准确无误；

4. 系统验证 74LS181 芯片的 16 种算术运算和 16 种逻辑运算功能，能够准确设置控制信号和输入运算数，如实记录实验结果，并对结果进行分析，验证芯片功能的正确性。

三、总体设计

1. 基本原理

(1) 运算类型控制：M=1 时，芯片工作于逻辑运算模式，运算结果仅与输入数据 A、B 相关，不受进位影响；M=0 时，工作于算术运算模式，运算结果受进位输入 Cn（Cn=0 为有进位，Cn=1 为无进位）影响。

(2) 运算功能选择：S3、S2、S1、S0 的 16 种组合对应 16 种运算（算术或逻辑），例如 S3S2S1S0=0000 时，逻辑运算为 F=~A，算术运算（Cn=1）为 F=A，算术运算（Cn=0）为 F=A+1。

(3) 数据输入与输出：A3~A0 和 B3~B0 为 4 位输入运算数引脚（引脚 3 为最高位），F3~F0 为 4 位运算结果输出引脚（F3 为最高位），Cn+4 为芯片产生的进位输出引脚（Cn+4=0 表示有进位输出，Cn+4=1 表示无进位输出）。

2. 模块介绍

(1) 电源与接地模块：提供 5V VCC 电源和接地端，确保芯片正常工作。

(2) 控制信号模块：由三组按键组成，分别控制运算类型（M）、运算选择（S3~S0）和进位输入（Cn）。每组按键通过按压与释放产生低电平和高电平信号，实现控制信号的电平切换，进而选择不同的运算模式和运算功能。

(3) 运算数输入模块：通过按键或拨码开关设置输入运算数 A、B（本实验采用固定数据 AH、5H、FH 等）。

(4) ALU 核心模块：以 74LS181N 芯片为核心，是整个实验电路的运算中心。其功能是接收控制信号模块的控制信号和运算数输入模块的输入运算数，按照预设的逻辑完成算术运算或逻辑运算，输出运算结果 F3~F0 和进位信号 Cn+4。

(5) 结果显示模块：DCD_HEX 数码管，用于显示运算结果 F3-F0（十六进制格式）

四、详细设计

1.电路结构

本实验电路基于 Multisim 平台搭建，采用单总线结构，各模块的连接关系和信号传输路径如图 1.1 所示：

解析电路模块（图 1.1）：

(1) 电源模块的 5V VCC 接入 74LS181N 芯片的 VCC 引脚（通常为引脚 20），芯片的 GND 引脚（通常为引脚 10）连接电路公共地，为芯片提供工作电源；其他辅助电路（如控制信号模块、显示模块）也分别接入 VCC 和 GND，确保正常工作。

(2) 控制信号模块的三组按键分别连接至 74LS181N 芯片的 M 引脚（运算类型控制）、S3~S0 引脚（运算选择控制）和 Cn 引脚（进位输入控制）。按键未按压时输出高电平，按压时输出低电平，通过切换按键状态实现控制信号的电平设置。

(3) 运算数输入模块的拨码开关或按键分别连接至 74LS181N 芯片的 A3~A0 引脚（运算数 A 输入）和 B3~B0 引脚（运算数 B 输入），通过调整拨码开关或按键状态，设置实验要求的输入数据（AH、5H 等）。

(4) 74LS181N 芯片的 F3~F0 引脚（运算结果输出）分别连接至 DCD_HEX 数码管的 4 个输入引脚，数码管将 4 位二进制结果转换为十六进制格式并显示；芯片的 Cn+4 引脚（进位输出）连接至一个指示灯，当有进位输出（Cn+4=0）时指示灯点亮，无进位输出（Cn+4=1）时指示灯熄灭。

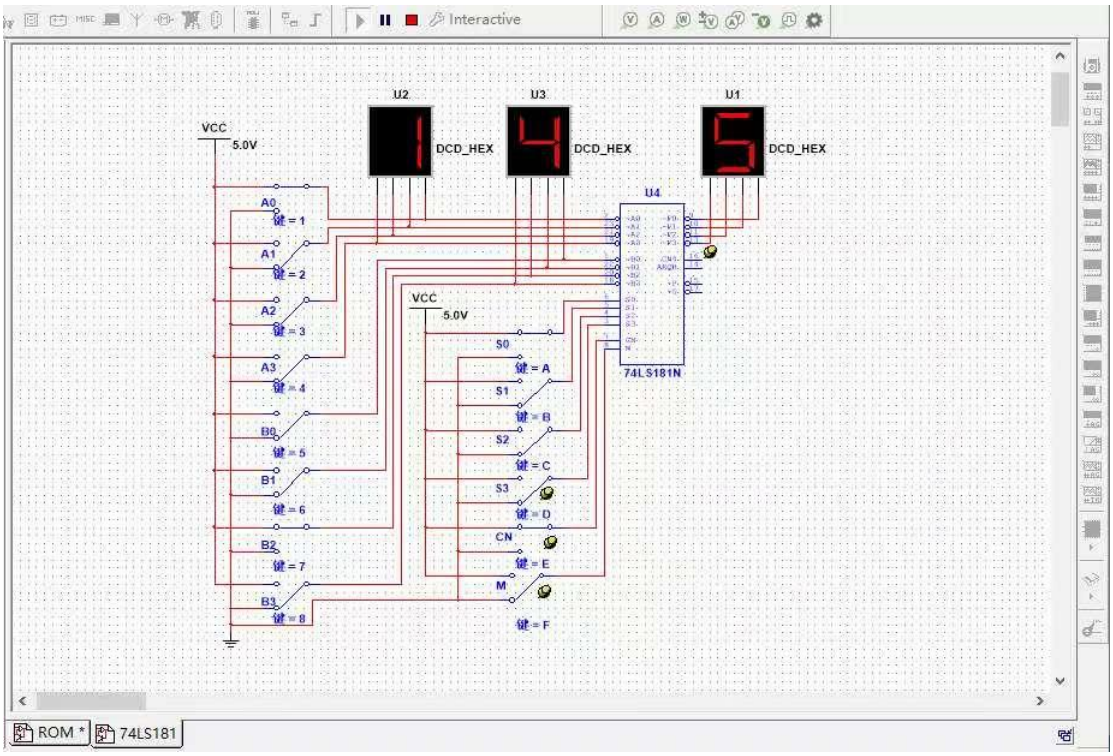


图 1.1 验证 74LS181 运算和逻辑功能电路图

2.具体模块设计

(1) 按实验表格要求，设置第一组 S0~S3 编码（如 0000），并配置对应输入数据（如 AH、5H）

(2) 先置 M=0（算术运算模式），分别设置 Cn=1（无进位）和 Cn=0（有进位）；再置 M=1（逻辑运算模式），保持 Cn 无效

(3) 数据输入：通过拨码开关 / 按键设置两组 4 位输入数据

(4) 进位运算：74LS181 芯片，接收输入数据和控制信号，完成算术 / 逻辑运算并输出结果 F3~F0，同时生成进位信号 Cn+4。

(5) 显示模块：采用数码管（DCD_HEX）将 4 位二进制结果转换为十六进制显示，便于读取和记录

3.运算结果读取与记录

(1)每完成一组参数配置，观察数码管显示的结果，将十六进制结果填入实验表格对应位置

(2)依次切换 S0~S3 的 16 种编码，重复 “配置 - 读取 - 记录” 流程，覆盖表格中所有测试项

4.实验原理:

由于在验证 74LS181 运算和逻辑功能时需要给 74LS181 的 S3~S0、M 和 CN

管脚分配相应的电平信号，所以在实验时可以根据下面表1.1 74LS181 的运算逻辑表分配所要验证的运算或逻辑功能。同时为了方便了解 74LS181 的引脚和作用，可以在表 1.2 ALU-74LS181 引脚说明表中了解相应的各引脚的名称和说明。

表 1.1 74LS181 的运算逻辑表

方式	M = 1 逻辑运算	M = 0 算术运算	
S3 S2 S1 S0	逻辑运算	CN=1 (无进位)	CN =0 (有进位)
0 0 0 0	F=~A	F=A	F=A 加 1
0 0 0 1	F=~(A + B)	F=A + B	F=(A + B) 加 1
0 0 1 0	F=(~A) B	F=A + ~B	F=(A + ~B)加 1
0 0 1 1	F=0	F=负 1（补码形式）	F=0
0 1 0 0	F=~(A B)	F=A 加 A (~ B)	F=A 加 A ~ B 加 1
0 1 0 1	F=~B	F=(A + B) 加 A ~B	F=(A + B)加 A ~ B 加 1
0 1 1 0	F=A B	F=A 减 B 减 1	F=A 减 B
0 1 1 1	F=A/B	F=A (~ B)减 1	F=A (~B)
1 0 0 0	F=~A +B	F=A 加 A B	F=A 加 A B 加 1
1 0 0 1	F=~(A B)	F=A 加 B	F=A 加 B 加 1
1 0 1 0	F=B	F=(A + ~B)加 A B	F=(A + ~B)加 A B 加 1
1 0 1 1	F=AB	F=AB 减 1	F=AB
1 1 0 0	F=1	F=A 加 A	F=A 加 A 加 1
1 1 0 1	F=A +~B	F=(A + B) 加 A	F=(A + B) 加 A 加 1
1 1 1 0	F=A + B	F=(A + ~B) 加 A	F=(A + ~ B) 加 A 加 1
1 1 1 1	F=A	F=A 减 1	F=A

表 1.1 详细列出了 S3~S0 的 16 种组合对应的逻辑运算（M=1）和算术运算

（M=0，含 Cn=1 无进位、Cn=0 有进位两种情况）功能，是实验中选择运算类型、验证运算结果的核心依据。实验时需根据该表确定 S3~S0 的组合的运算功能，再对比实际读出结果与理论结果是否一致。

5. ALU-74LS181 引脚说明：M=1 逻辑运算，M=0 算术运算

表 1.2 明确了 74LS181N 芯片关键引脚的功能，包括运算类型控制端（M）、运算选择控制端（S3~S0）、运算数输入端（A3~A0、B3~B0）、进位输入 / 输出端（Cn、Cn+4）和运算结果输出端（F3~F0），是电路接线和参数设置的重要参考。

表 1.2 ALU-74LS181 引脚说明表

引　脚	说　　明
M 状态控制端	M=1 逻辑运算；M=0 算术运算。
S3 S3 S1 S1 运算选择控制	S3 S3 S1 S1 决定电路执行哪一种算术
A3 A2 A1 A1	运算数 1，引脚 3 为最高位
B3 B2 B1 B0	运算数 2，引脚 3 为最高位
Cn　最低位进位输入	Cn =0 有进位；Cn =1 无进位；
Cn+4　本片产生的进位信号	Cn+4=0 有进位；Cn+4=1 无进位；
F3 F2 F1 F0	F3 F2 F1 F0 运算结果，F3 为最高位

五、实验结果与分析

最终实验结果可以按照表 1.3 来验证电路的正确性，电路的实验结果如表 1.3 所示：

表 1.3 以 “控制信号（S3~S0）+ 输入数据（数据 1、数据 2）” 为变量，对应记录 3 类运算结果：无进位算术运算（M=0，CN=1）、有进位算术运算（M=0，CN=0）、逻辑运算（M=1），共设置 16 组测试用例（覆盖 S3~S0 全组合）。

表 1.3 实验结果表

S3 S2 S1 S0	数据 1	数据 2	算术运算（M=0）		逻辑运算（M=1）
			CN=1 （无进位）	CN =0 （有进位）	
0 0 0 0	AH	5H	F=AH	F=BH	F=5H

0 0 0 1	AH	5H	F=FH	F=0H	F=0H
0 0 1 0	AH	5H	F=AH	F=BH	F=5H
0 0 1 1	AH	5H	F=FH	F=0H	F=0H
0 1 0 0	FH	1H	F=DH	F=EH	F=EH
0 1 0 1	FH	1H	F=DH	F=EH	F=EH
0 1 1 0	FH	1H	F=DH	F=EH	F=EH
0 1 1 1	FH	1H	F=DH	F=EH	F=EH
1 0 0 0	FH	FH	F=EH	F=FH	F=FH
1 0 0 1	FH	FH	F=EH	F=FH	F=FH
1 0 1 0	FH	FH	F=EH	F=FH	F=FH
1 0 1 1	FH	FH	F=EH	F=FH	F=FH
1 1 0 0	5H	5H	F=AH	F=BH	F=FH
1 1 0 1	5H	5H	F=AH	F=BH	F=FH
1 1 1 0	5H	5H	F=4H	F=5H	F=5H
1 1 1 1	5H	5H	F=4H	F=5H	F=5H

六、小结与心得体会

1. 本次实验通过搭建 74LS181 功能验证电路，系统验证了该芯片的 16 种算术运算和 16 种逻辑运算功能，让我对算术逻辑单元（ALU）的工作原理有了更深入、更直观的认识，也提升了自己的电路设计、操作和数据分析能力。

2. 通过实验一，我熟练掌握了 74LS181 芯片的使用方法，理解了算术运算与逻辑运算的实现机制，熟悉了简单运算器的数据传送通路，为后续运算器综合实验和微程序控制实验奠定了良好的基础。

3. 实验过程中遇到的主要问题是初期对 S3-S0 组合对应的运算功能记忆不清晰，导致数据记录混乱，通过对照功能表逐步验证，最终理清了各组合的逻辑关系。同时，我也深刻体会到控制信号的精确性对实验结果的重要性，任何一个按键的接触不良都会导致结果错误。

实验 2 运算器（2）

一、实验名称

运算器（2）

二、实验目的

1.熟练掌握算术逻辑单元（ALU）在复杂电路中的应用方法，理解 ALU 作为运算核心与其他模块的协同工作机制；

2.进一步深入理解简单运算器的单总线数据传送原理，明确数据在各功能模块间的传输路径和控制逻辑；画出逻辑电路图及布出美观整齐的接线图；

3.具备独立绘制单总线结构运算器逻辑电路图和规范接线图的能力，能够合理布局元件、规划总线和控制信号路径；

4.熟练掌握 74LS244N、74LS273N、74LS374N 等关键数字元件的功能和使用方法，明确其在运算器中的具体作用；

三、总体设计

1.基本原理

本实验仿真的单总线结构运算器以 8 位数据总线为核心，各模块功能如下：

(1) 数据输入模块（K8 层次块）：产生 8 位输入数据，为运算提供原始数据。

(2) 三态门模块（74LS244N 层次块）：通过使能信号控制模块与总线的连接或断开，确保同一时间总线上只有一个模块向总线输出数据，避免数据冲突。

(3) 暂存器模块（74LS273N 层次块，DR1、DR2）：临时存储从总线传送的运算数，为 ALU 提供稳定的输入，避免因总线数据变化导致运算出错。

(4) 通用寄存器模块（74LS374N 层次块，GR1、GR2）：存储运算过程中的数据和结果，可实现数据的读写控制。

(5) ALU 模块（8 位 ALU 层次块）：运算核心，接收暂存器传送的运算数，完成指定的算术或逻辑运算，输出运算结果。

(6) 显示模块（8 段数码管）：实时显示数据总线、寄存器、ALU 输出等关键节点的数据信息。

(7) 控制模块（开关、按键）：提供控制电平和打入脉冲，控制三态门的通断、寄存器的读写、ALU 的运算类型选择等，确保整个运算过程按顺序有序进行。

2.模块介绍

(1) 8 位数据总线：由 8 条导线组成，连接 K8、74LS244N、74LS273N、74LS374N、8 位 ALU 的数据线，是数据传送的核心通道。

(2) 控制信号：包括三态门使能信号（G）、暂存器时钟信号（CLK）、通用寄存器时钟信号（CLK）和输出使能信号（OC）、ALU 运算控制信号等，通过开关或按键手动控制。

四、详细设计

1.74LS244N 在电路中的作用：

三态输出的八缓冲器 / 线驱动器，具备信号缓冲、驱动和三态控制功能，能够增强信号的驱动能力，同时通过三态控制实现模块与总线的隔离。

2.74LS273N 在电路中的作用：

带清除端的 8 位上升沿触发 D 触发器，具备数据锁存功能，能够在时钟信号的上升沿将输入数据锁存至输出端，保持输出数据稳定，不受输入数据后续变化的影响。

3.74LS374N 的功能及其在电路中的作用：

8 位三态输出 D 触发器，具备数据锁存和三态控制功能，既能在时钟信号的上升沿锁存输入数据，又能通过三态控制实现输出端与外部电路的隔离。

4.8 位 ALU 层次块：

由两片 74LS181N 芯片级联而成，实现 8 位算术运算（加、减）和逻辑运算（与、或、异或、非），运算类型由控制信号 M 和 S0~S4 决定。

5.具体模块设计：

(1) 缓冲驱动模块：由多个 74244 模块组合而成，每片 74244 负责 8 路信号的缓冲，输入接按键模块，输出接后续的存储 / 显示模块；利用 74244 的三态特性，实现多组信号的 “无冲突并行传输”

(2) 锁存模块：由多片 74373 芯片组成，临时存储按键输入的二进制信号，保证显示稳定，当使能端有效时，输入信号直接传到输出；使能端无效时，保持当前输出状态，避免按键松开后信号立即消失，让数码管显示更稳定；

(3) 显示输出模块：多组 DCD_HEX 共阴极七段数码管组成，每组 2 个数码管对应 8 位二进制（2 位十六进制），通过内置的解码电路，将 4 位二进制信号转换为 1 位十六进制的段码显示；6 组数码管共显示 12 位十六进制，对应 48 路按键输入的状态

五、实验结果与分析

1.总体设计图以及部分器件的组成设计

(1) 运算器总体设计图如图 2.1 所示：

主要构成：包含输入端口（IN5、IN6、IN7、IN8 等）、输出端口（OUT1-OUT8）、时钟信号端（CLK）、电源端（VCC）、清零端（CLR）以及核心逻辑器件连接线路，整体呈现运算器各模块的互联架构。

功能：作为运算器的整体框架图，明确各输入输出信号的传输路径、核心器件的联动关系，为后续算术运算（加、减）和逻辑运算（与、或、异或）提供硬件连接基础，是整个运算系统的拓扑核心。

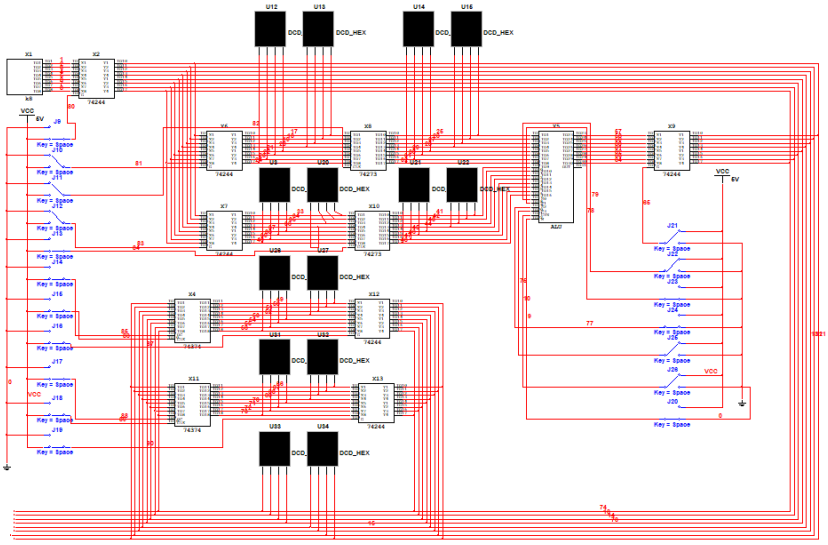


图 2.1 运算器总体设计图

（2）K8_BLOCK 模块实现的电路能实现总线上的数据输入，按键 J8~J1 控制输入数字的电平信号，电路中的数码管显示屏从左往右，J8~J5 控制右边的数码管数字的显示，其余的控制左边的数码管的数字显示。

（3） 8bit_ALU 由 2 片 4 位的 74LS181N 级联而成：两片共享 M（运算类型）S3~S0（运算选择）控制信号，保证运算逻辑一致；低 4 位运算数接第一片，高 4 位接第二片；第一片的进位输出连第二片的进位输入，实现 8 位运算的进位传递；两片输出组合为 8 位运算结果。

2. K8 产生任意数据存入通用寄存器 GR1

流程说明：实现“数据生成 - 存储”功能通过 K8 模块产生任意 8 位数据，在写入控制信号的触发下，将生成的数据传输并存储到通用寄存器 GR1 中，为后续运算提供原始数据输入。实现 K8 产生任意数据存入通用寄存器 GR1 功能如图 2.2 所示：

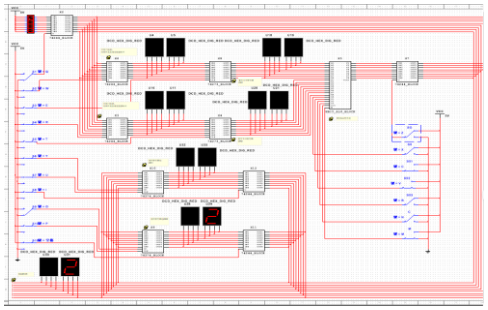


图 2.2　GR1 寄存器存储图

3. K8 产生任意数据存入通用寄存器 GR2

流程说明：与图 2.7 功能对称，通过 K8 模块生成另一路任意 8 位数据，经控制信号触发存入通用寄存器 GR2，为运算提供第二路原始数据，与 GR1 的数据配合完成各类运算。实现 K8 产生任意数据存入通用寄存器 GR2 功能如图 2.3 所示：

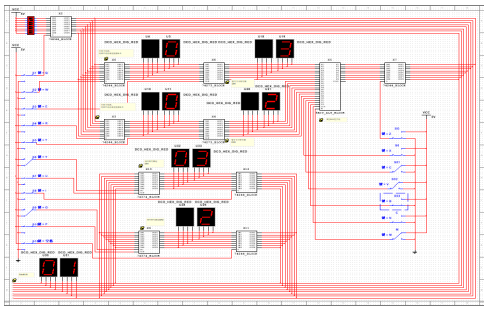


图 2.3　GR2 寄存器存储图

4. 完成 GR1+GR2→GR1

流程说明：实现 8 位算术加法运算。ALU 接收 GR1 和 GR2 存储的数据，在加法控制信号作用下执行加法运算，运算结果通过反馈线路覆盖存入 GR1，完成 “GR1 = GR1 + GR2” 的运算逻辑。

实现 GR1+GR2→GR1 功能如图 2.4 所示：

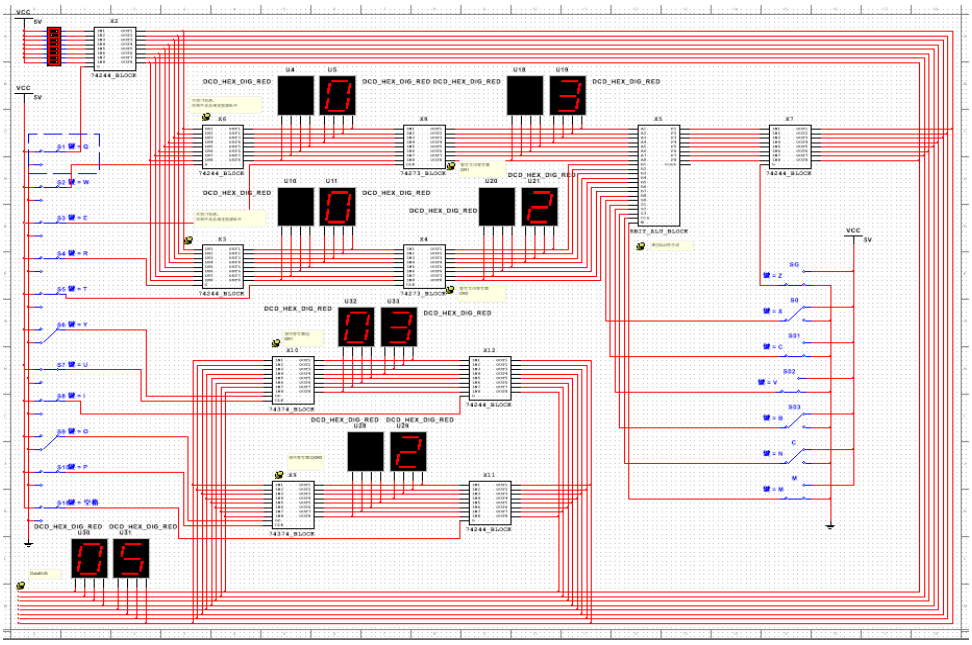


图 2.4　加法运算图

5.完成 GR1-GR2→GR1

流程说明：实现 8 位算术减法运算。ALU 在减法控制信号作用下，对 GR1 和 GR2 的数据执行减法运算（通常通过补码加法实现），运算结果反馈存入 GR1，完成 “GR1 = GR1 - GR2” 的逻辑功能。

实现 GR1-GR2→GR1 功能如图 2.5 所示：

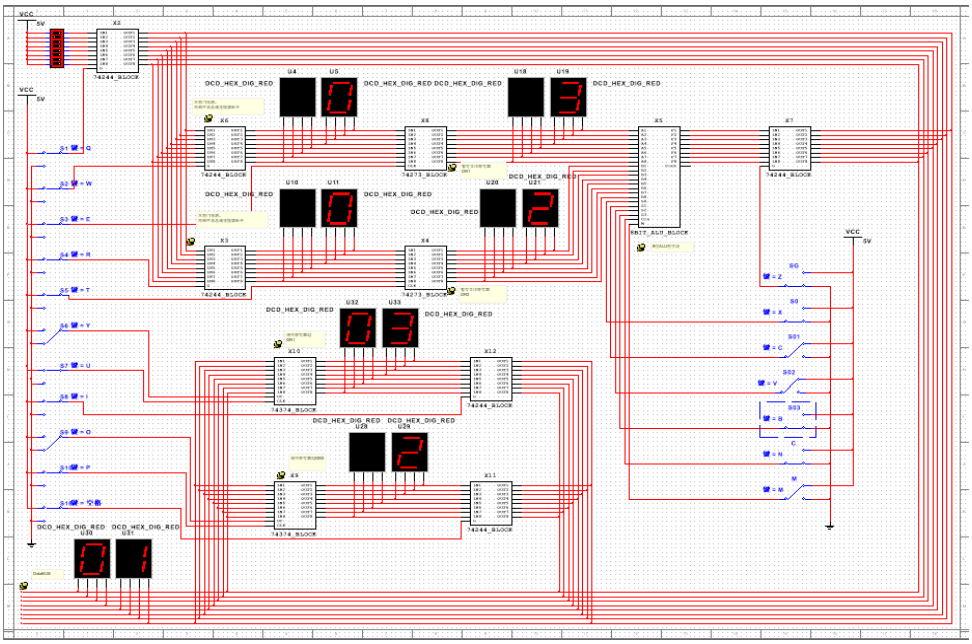


图 2.5 减法运算图

6.完成 GR1∧GR2→GR1

流程说明：实现 8 位逻辑 “与” 运算。ALU 接收 GR1 和 GR2 的数据，在 “与” 控制信号触发下，对对应位执行 “全 1 出 1，有 0 出 0” 的逻辑运算，结果写入 GR1，完成 “GR1 = GR1 ∧ GR2” 的功能。

实现 GR1∧GR2→GR1 功能如图 2.6 所示

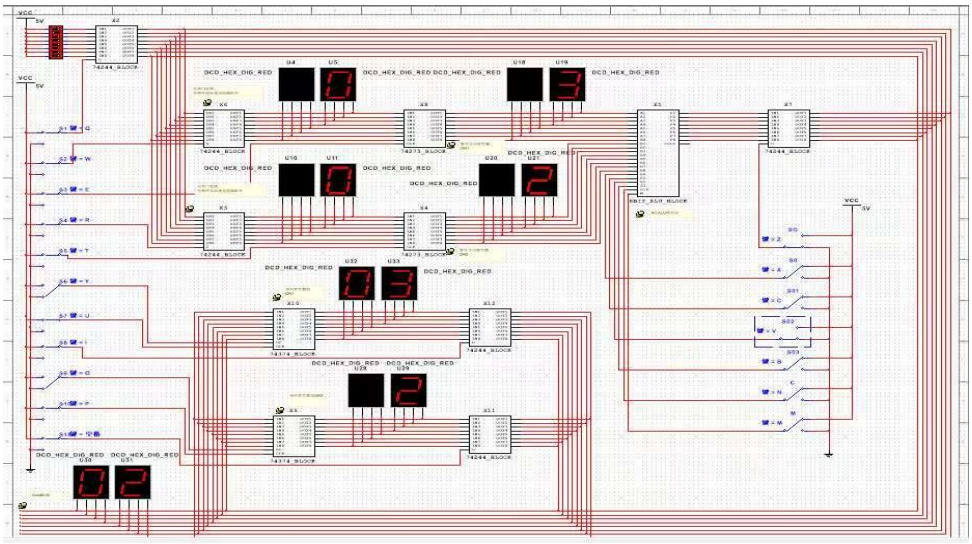


图 2.6　与运算图

7.完成 GR1∨GR2→GR2

流程说明：实现 8 位逻辑 “或” 运算。ALU 对 GR1 和 GR2 的对应位执行 “有 1 出 1，全 0 出 0” 的逻辑运算，在 “或” 控制信号作用下，将运算结果写入 GR2，完成 “GR2 = GR1 ∨ GR2” 的逻辑功能

实现 GR1∨GR2→GR2 功能如图 2.7 所示

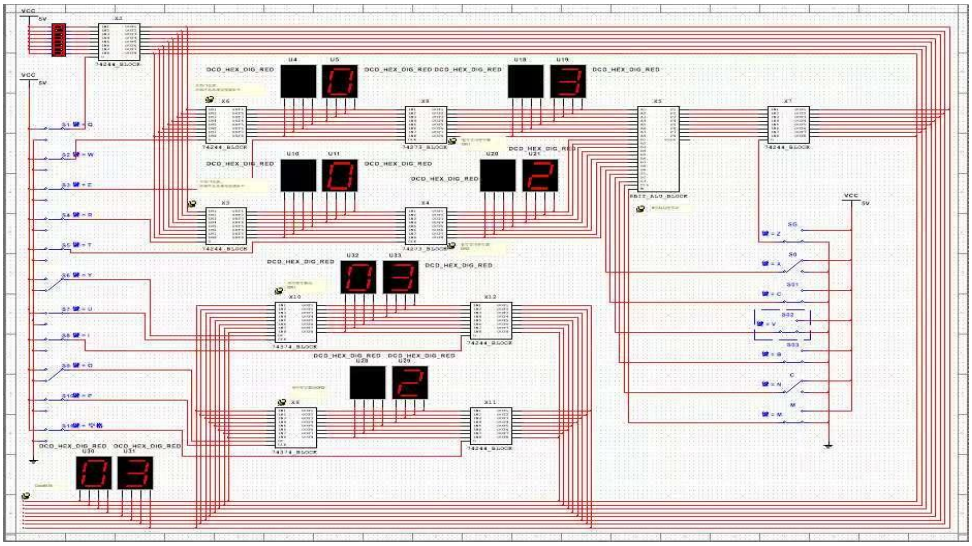


图 2.7　或运算图

8.完成 GR1⊕ GR2→GR1

流程说明：实现 8 位逻辑 “异或” 运算。ALU 对 GR1 和 GR2 的对应位执行 “不同出 1，相同出 0” 的逻辑运算，运算结果通过控制信号写入 GR1，完成 “GR1 = GR1 ⊕ GR2” 的功能，常用于数据比较、校验等场景。

实现 GR1⊕ GR2→GR1 功能如图 2.8 所示：

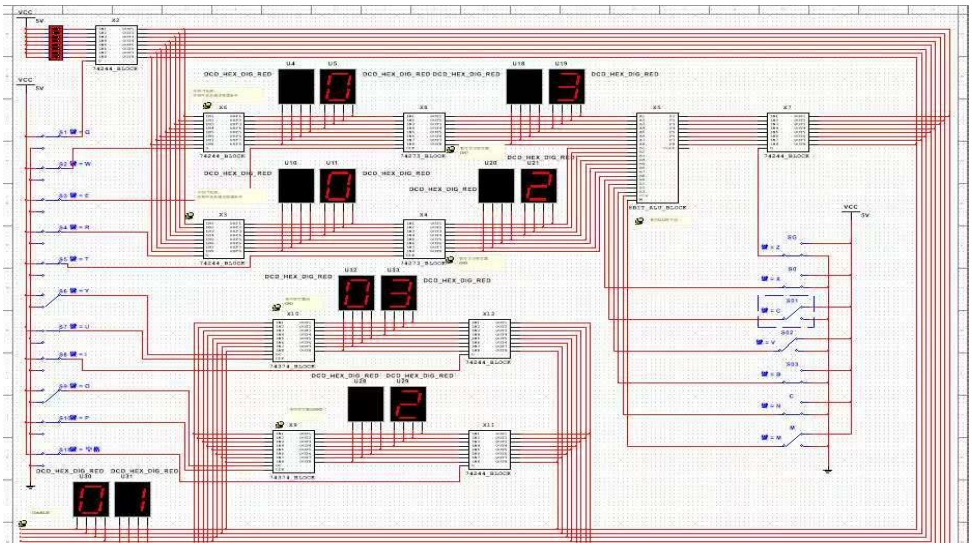


图 2.8　异或运算图

9.结果分析

(1) 数据存储功能验证：通过 K8 成功将数据写入 GR1 和 GR2，数码管显示与输入数据一致，证明 74LS374N 的写入功能正常，CLK 信号的触发逻辑正确。

(2) 数据传送功能验证：GR1、GR2 的数据通过总线成功传送至 DR1、DR2，ALU 接收数据后能正确输出运算结果，证明三态门(74LS244N)的通断控制有效，总线数据传送无冲突，暂存器（74LS273N）的锁存功能可靠。

(3) 运算功能验证：所有算术运算和逻辑运算结果均与理论计算值一致。

(4) 电路稳定性验证：多次重复上述操作，结果始终一致，无数据丢失或误码现象，证明单总线结构运算器的设计合理，各模块协同工作稳定。

六、小结与心得体会

1．本次单总线结构运算器实验是对计算机组成原理知识的综合应用，实验难度较大，但通过本次实验，我不仅深入理解了运算器的内部结构和工作机制，还熟练掌握了多种关键数字元件的功能和使用方法，提升了复杂电路的设计、调试和故障排查能力。

2．在实验设计阶段，子电路（层次块）的创建是重点和难点。通过反复尝试和查阅 Multisim 操作手册，我成功创建了 K8、74LS244N、74LS273N、74LS374N 和 8 位 ALU 等层次块，学会了如何合理设置输入输出引脚、布局元件和接线，这一过程不仅提升了我的软件操作能力，还培养了我的模块化设计思维 —— 将复杂电路拆分为多个简单子电路，降低了整体设计难度，提高了电路的可读性和可维护性。

3．通过完成各项运算操作，我进一步熟悉了 ALU 的运算原理和数据传送路径，理解了暂存器、通用寄存器在运算过程中的作用，明确了控制信号（如 CLK、G、OC）对电路工作状态的影响。通过本次实验，我不仅巩固了计算机组成原理的理论知识，还提升了实践操作能力和解决实际问题的能力，为后续微程序控制等更复杂的实验奠定了坚实的基础。

实验 3 字发生器及跑马灯

一、实验名称

字发生器跑马灯。

二、实验目的

1. 深入了解 Multisim 平台中字信号发生器（Word Generator）的核心功能和工作原理，熟练掌握其参数设置方法（如输出模式、数据变化规律、显示格式、频率、触发方式等）；

2. 能够设计跑马灯电路，实现字发生器与外部显示电路的协同工作，通过字发生器输出的信号控制探针依次点亮，形成流畅的跑马灯效果；

3. 提升电路设计、参数调试和结果分析能力，理解时序信号频率对显示效果的影响，掌握实验参数的优化方法。

三、总体设计

1.基本原理

(1) 字发生器工作原理:字发生器可产生 32 位并行数字信号(输出端 0~31)，通过设置输出模式（Cycle、Burst、Step）、数据变化规律（Up Counter、Down Counter、Shift 等）、数据格式（Hex、Dec、Binary 等）和输出频率，可产生符合要求的同步时序信号。本实验采用高 16 位输出（16~31），控制 16 个探针。

(2) 跑马灯实现原理:将字发生器的高 16 位输出端分别连接至 16 个探针，通过设置字发生器的数据，使每个输出端按顺序产生高电平，对应的探针依次点亮，形成跑马灯效果。

四、详细设计

1.电路结构

本实验的跑马灯电路结构如图 3.1 所示，各模块的连接关系和信号传输路径如下：

1. 字发生器模块：调用 Multisim 中的字信号发生器（路径：Simulate→Instruments→Word Generator），放置于电路工作区。字发生器的

左侧输出端为 0~15（低 16 位），右侧输出端为 16~31（高 16 位），本实验仅使用左侧 0~15 输出端；字发生器的电源引脚接入 5V VCC，接地引脚连接公共地；R 端（时钟脉冲输出端）和 T 端（外部触发输入端）本次实验未使用，悬空处理。

2. 显示模块：在电路工作区放置 16 个探针，分别命名为 X0~X15。将探针 X0 的输入端连接字发生器的 0 号输出端，X1 连接 1 号输出端，……，X15 连接 15 号输出端，实现字发生器低 16 位输出端与探针的一对一连接。

3. 电源与接地模块：将 5V VCC 电源的正极连接字发生器的 VCC 引脚，负极连接公共地；字发生器的 GND 引脚也连接公共地，确保电路电平参考一致。

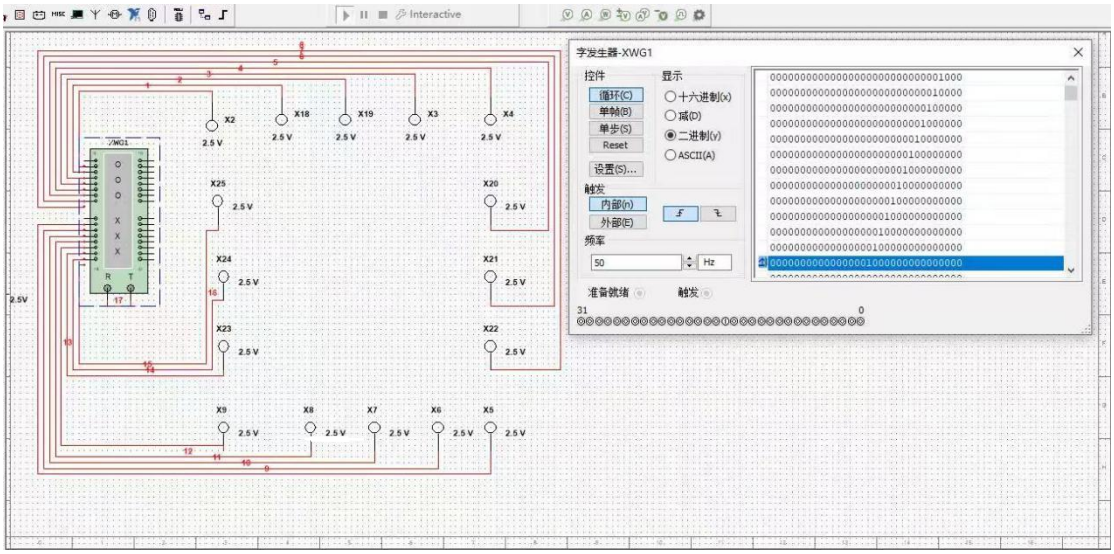


图 3.1 字发生器及跑马灯图

2.字发生器参数设置

(1) Control（输出模式）设置：选择 “Cycle” 模式，该模式下字发生器将在预设的初始值和终止值之间循环输出数字信号，确保跑马灯效果持续循环。

(2) Set（数据变化规律）设置：点击 “Set” 按钮，弹出数据变化规律设置对话框，选择 “Shift Left”（左移）模式，设置初始 Pattern（初始数据）为 0000000000000001（二进制），即低 16 位中仅 0 号输出端为高电平，其余为低电平；设置 Buffer Size（缓冲区长度）为 0x0010（16），即循环输出 16 组数据，对应 16 个探针的依次点亮。

(3) Display（显示格式）设置：选择 “Binary”（二进制）格式，便于直观观察字发生器输出数据的变化情况，确保数据按预设规律变化。

(4) Trigger（触发方式）设置：选择 “Internal”（内部触发）模式，字发生器的信号输出由自身的输出模式控制，无需外部触发信号，简化电路设计。

(5) Frequency（输出频率）设置：将频率调整为 50Hz，该频率下跑马灯的点亮速度适中，便于观察跑马效果；后续可根据实际效果调整频率（如 100Hz 加快速度，25Hz 减慢速度）。

(6) 参数保存：完成上述参数设置后，点击“Save”按钮，将参数保存为“跑马灯参数.dp” 文件，避免后续实验重复设置，下次实验可直接通过“Load”按钮载入参数。

五、实验结果与分析

1. 实验现象

仿真启动后，探针从 X0 到 X15 依次点亮，循环往复，形成流畅跑马灯效果，载入保存参数文件可直接复现效果。点击“开始运行”后，现象如图 3.2、图 3.3 所示：

(1) 运行状态展示：呈现仿真启动后的跑马灯实际运行效果，探针从 X0 到 X15 依次点亮，循环往复，形成流畅的动态切换效果；

(2) 信号验证功能：通过探针的亮灭状态，直观验证字发生器输出信号的有效性和有序性 —— 探针按递增顺序点亮，说明字发生器的 Up Counter 递增计数功能正常，信号传输无异常；

(3) 效果复现标识：该运行效果可通过载入保存的参数文件直接复现，无需重新配置参数，提升实验的可重复性和效率。

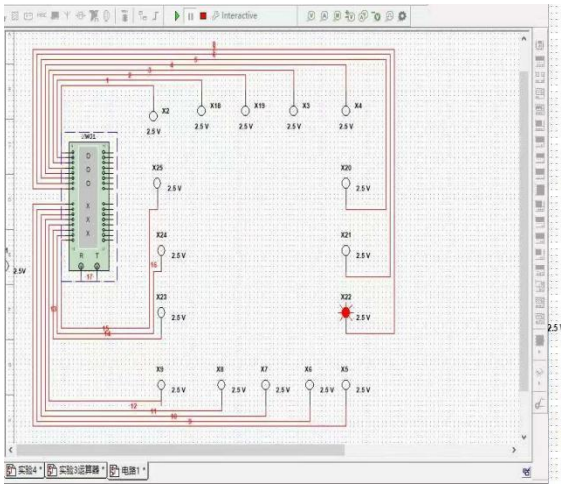


图 3.2 跑马灯图实验现象

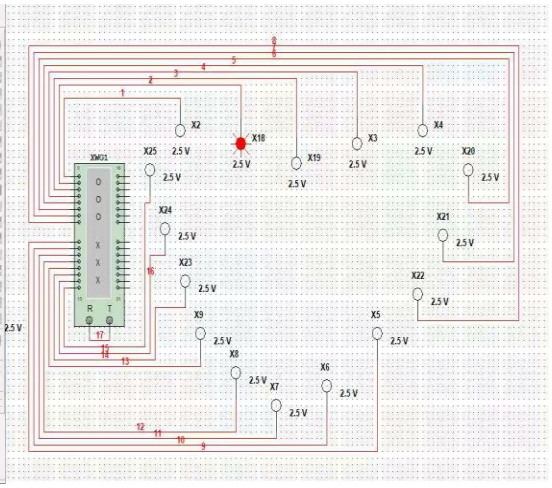


图 3.3 跑马灯图实验现象

2.结果分析

字发生器参数设置有效,信号传输稳定;频率调整可改变跑马速度(100Hz 加快、25Hz 减慢）；长时间仿真无乱序，电路可靠。

六、小结与心得体会

1．本次字发生器及跑马灯实验是一项相对简单但实用性较强的实验，通过实验我熟练掌握了 Multisim 平台中字信号发生器的使用方法,深入理解了时序信号的产生和应用原理，提升了电路设计和参数调试能力。

2．在实验过程中，我首先学习了字发生器的核心功能和参数设置方法，通过反复尝试不同的输出模式、数据变化规律和频率，逐渐理解了各参数对输出信号的影响。例如，在设置数据变化规律时，最初选择了 “Up Counter”（递增计数）模式，虽然也能实现跑马灯效果，但高电平的移动逻辑不如 “Shift Left”模式直观，最终选择了更贴合跑马灯需求的 “Shift Left” 模式。

实验 4 模拟微程序实现指令

一、实验名称

模拟微程序实现指令

二、实验目的

1．模拟微程序实现机器语言指令,深入理解微程序控制的基本原理，掌握微指令、微程序与机器语言指令的关系。

2．学会使用字发生器模拟微指令，取代手动开关控制，实现机器语言指令的自动执行。

3．熟练掌握单总线结构运算器与字发生器的协同工作机制，实现指定的算术和逻辑运算指令。

三、总体设计

1.基本原理

(1) 微程序控制核心逻辑：机器语言指令的执行依赖若干条微指令的有序组合（即微程序）。每条微指令对应一组控制信号，用于驱动硬件模块（如运算器、寄存器、总线）完成特定操作；若干条微指令按顺序执行，即可实现一条完整的机器语言指令。

(2) 字发生器模拟微指令：字发生器的一行输出数据对应一条微指令，其 32 位并行输出端可直接映射为运算器的控制信号（如三态门使能、寄存器时钟、ALU 运算选择等）。通过设置字发生器的数据变化规律（如 Cycle 循环、Step 单步），可实现微指令的连续输出，进而驱动运算器自动执行指令。

(3) 与单总线运算器协同机制：沿用实验 2 的单总线结构运算器（核心模块包括 8 位数据总线、K8 数据输入模块、74LS244N 三态门、74LS273N 暂存器、74LS374N 通用寄存器、8 位 ALU），字发生器输出的微指令信号直接接入运算器的控制端，取代原手动开关，实现控制信号的自动时序输出，完成数据存储、运算及结果写入的全流程自动化。

2.模块介绍

(1) 微指令生成模块（字发生器）：核心功能是按预设微程序输出 32 位微指令（控制信号）。字发生器的 32 个输出端映射为运算器的各类控制信号，通过设置输出模式（Cycle）、数据变化规律（按微程序顺序编排）和频率，实现微指令的连续、有序输出。

(2) 运算器核心模块：沿用实验 2 的单总线结构运算器，包括 8 位数据总线、K8 数据输入模块、74LS244N 三态门、74LS273N 暂存器（DR1、DR2）、74LS374N 通用寄存器（GR1、GR2）、8 位 ALU 模块和显示模块。核心功能是接收字发生器的微指令信号，完成数据存入、算术 / 逻辑运算、结果存储等操作。

(3) 控制信号映射模块：将字发生器的 32 位输出端与运算器的控制端建立一一映射关系，明确每一位输出对应的控制功能（如三态门使能 G、寄存器时钟 CLK、ALU 运算选择 S0~S4、运算类型 M 等），确保微指令信号能准确驱动运算器模块工作。

(4) 显示与验证模块：由 8 段数码管组成，实时显示通用寄存器（GR1、GR2）、ALU 输出、数据总线的关键数据，用于直观验证微程序执行结果是否符合预期。

四、详细设计

1. 电路结构与连接

(1) 实验采用了字发生器与单总线运算器的协同架构，如图 4.1 所示，核心模块包括字发生器（微指令生成）、单总线运算器（指令执行）、控制信号映射线路（32 位控制线）和显示模块（结果验证）。字发生器的 32 个输出端通过控制线直接接入运算器的对应控制端，取代原手动开关，实现控制信号自动输入。

(2) 运算器模块完全沿用实验 2 的单总线运算器硬件结构，包括 8 位数据总线、K8 数据输入模块、74LS244N 三态门、74LS273N 暂存器、74LS374N 通用寄存器（GR1、GR2）、8 位 ALU（两片 74LS181 级联）和数码管显示模块，无需额外新增硬件。

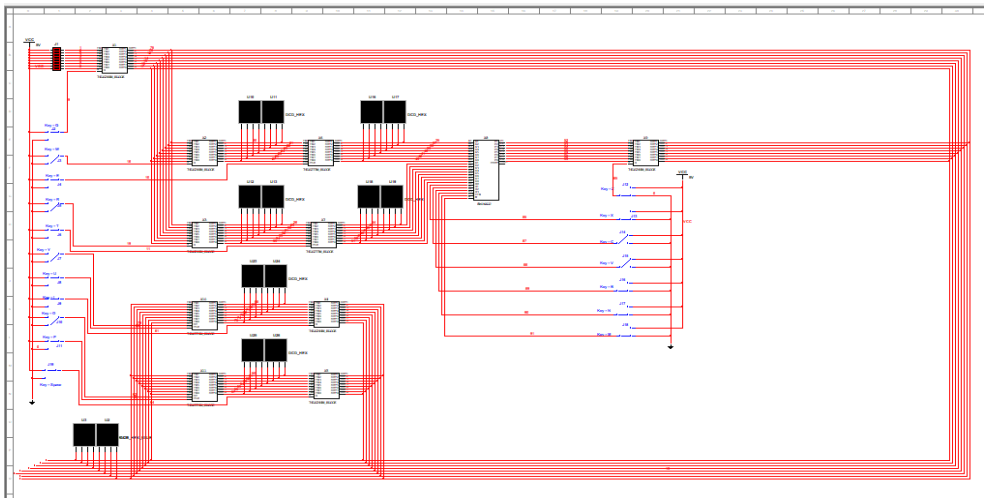


图 4.1 电路结构图

五、实验结果与分析

1.K8 产生任意数据存入通用寄存器 GR1

流程说明：

(1) 数据生成阶段：K8 数据产生模块根据设计逻辑，随机生成一组 8 位二进制原始数据（数据范围为 00000000~11111111，对应十进制 0~255），该数据为运算所需的第一路输入数据。

(2) 信号传输阶段：生成的 8 位数据通过预设的数据传输线路，精准传输至通用寄存器 GR1 的输入端口，传输过程中由 74LS244N 缓冲器增强信号驱动能力，保障数据传输的稳定性，避免信号衰减或干扰。

(3) 数据锁存阶段：系统提供同步时钟信号（CLK）和 GR1 专属写入控制信号，当写入控制信号有效且时钟信号触发时，GR1 内部的 74LS273N 寄存器启动锁存功能，将输入端口的 8 位数据稳定存储至寄存器内部，完成 “数据生成 - 传输 - 存储” 的全流程，为后续运算准备第一路数据。

实现 K8 产生任意数据存入通用寄存器 GR1 功能如图 4.2 所示：

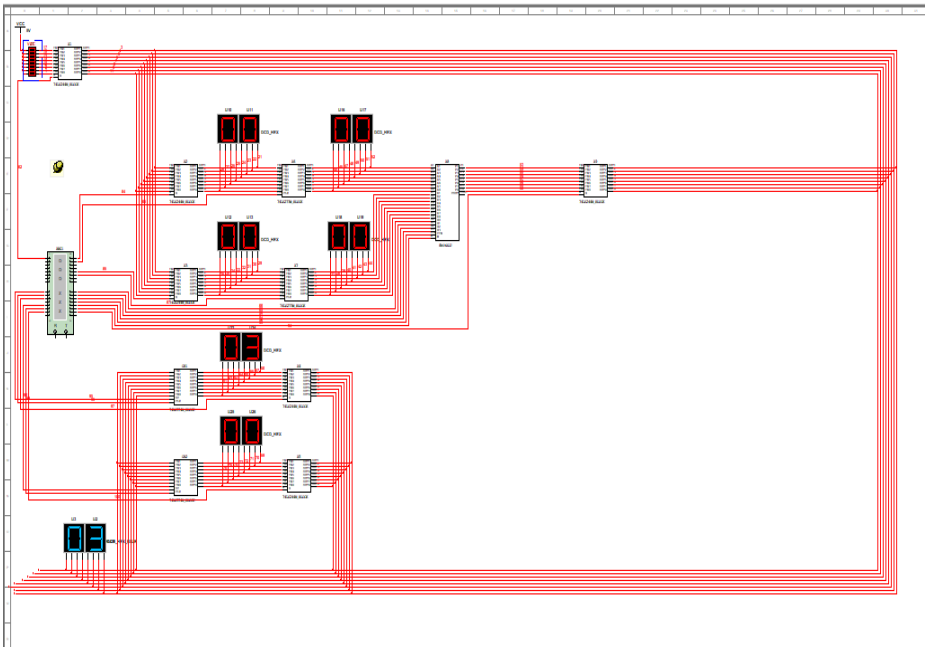


图 4.2 K8 产生任意数据存入通用寄存器 GR1

2.K8 产生任意数据存入通用寄存器 GR2

流程说明：

(1) 数据生成阶段：K8 数据产生模块独立生成另一组 8 位二进制任意数据（与 GR1 存储的数据可相同或不同，满足多样运算场景需求），作为运算所需的第二路输入数据。

(2) 信号传输阶段：该路数据通过独立的传输线路传输至通用寄存器 GR2 的输入端口，同样经过 74LS244N 缓冲器进行信号优化，确保数据在传输过程中保持完整性和准确性。

(3) 数据锁存阶段：在同步时钟信号（CLK）和 GR2 写入控制信号的协同作用下，GR2 内部的存储单元（74LS273N 或 74LS374）触发锁存动作，将接收的 8 位数据固定存储，完成第二路运算数据的预处理，此时 GR1 和 GR2 均存储有效数据，为后续算术运算（加、减）提供双路输入条件。

实现 K8 产生任意数据存入通用寄存器 GR2 功能如图 4.3 所示：

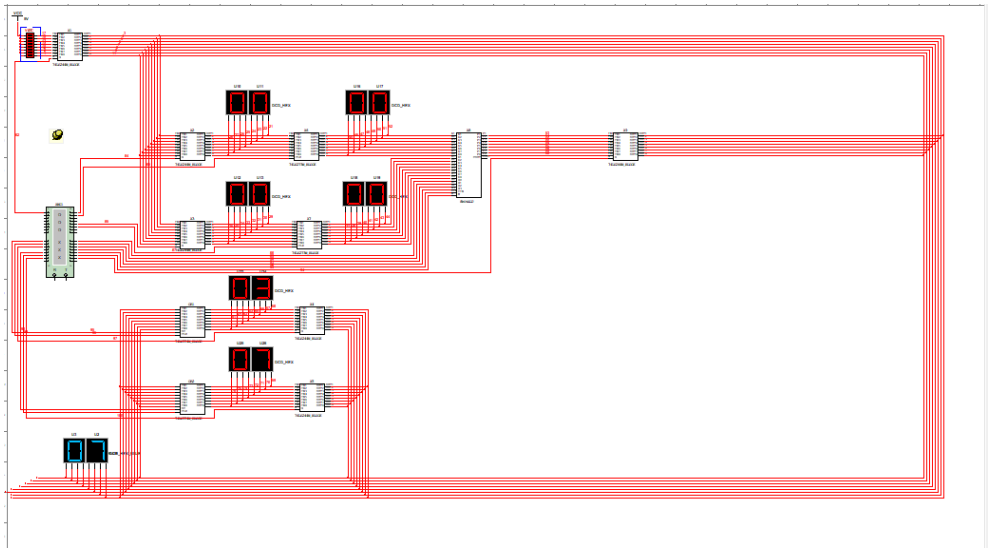


图 4.3 K8 产生任意数据存入通用寄存器 GR2

3.完成 GR1-GR2→GR1

流程说明：

(1) 数据读取阶段：系统发出运算启动信号，GR1 和 GR2 的输出控制信号有效，两者分别将存储的 8 位数据通过数据总线传输至 8bit_ALU 的两个输入端口（A 端口接收 GR1 数据，B 端口接收 GR2 数据）。

(2) 减法运算阶段：ALU 接收 “减法” 控制信号，激活内部减法运算单元，通过补码运算逻辑将减法操作转化为加法操作（即 GR1 数据 + GR2 数据的补

码），完成 8 位二进制数据的减法运算，生成对应的 8 位运算结果（含符号位，适配有符号数运算场景）。

(3) 结果存储阶段：减法运算结果通过 ALU 输出端口传输至 GR1 的输入端口，同时系统发出 GR1 更新控制信号，在时钟信号同步触发下，GR1 内部寄存器解锁并接收新数据，将原存储数据覆盖为减法运算结果，最终实现 “GR1 = GR1 - GR2” 的功能，运算结果留存于 GR1 供后续使用或输出。

实现 GR1-GR2→GR1 功能如图 4.4 所示：

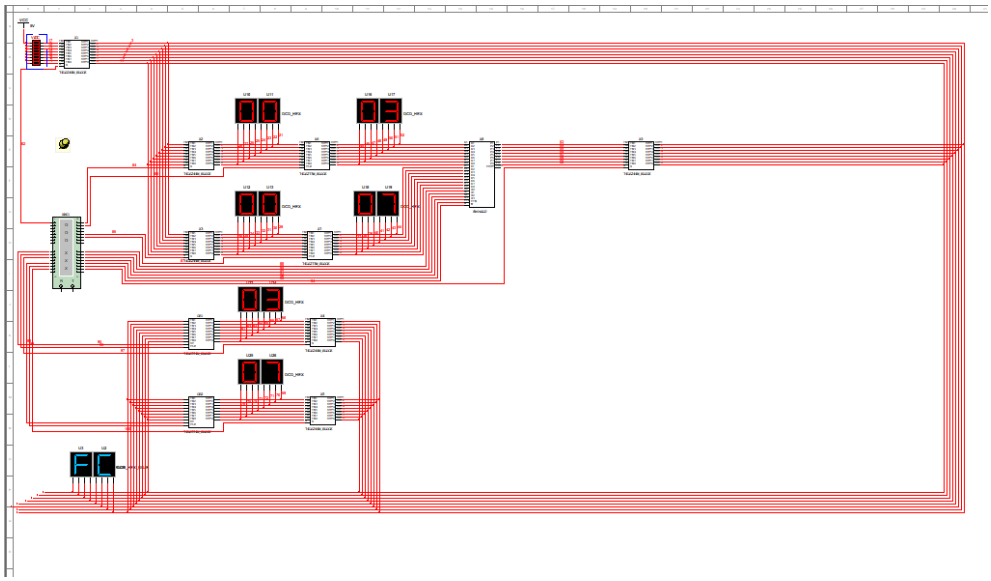


图 4.4 实现 GR1-GR2→GR1

4.完成 GR1+GR2→GR1，将减法的结果存入寄存器 1 中

流程说明：

(1) 数据读取阶段：运算控制信号触发后，GR1 和 GR2 的输出使能信号有效，两者存储的 8 位数据分别通过数据总线传输至 8bit_ALU 的 A、B 输入端口，数据传输过程由总线逻辑协调，避免冲突。

(2) 加法运算阶段：ALU 接收 “加法” 控制信号，启动内部加法运算单元，对 A 端口（GR1 数据）和 B 端口（GR2 数据）的 8 位二进制数据执行按位加法运算，处理进位信号，生成 8 位加法运算结果（含最终进位信息，适配无符号数 / 有符号数加法场景）。

(3) 结果存储阶段：加法运算结果通过反馈线路传输至 GR1 的输入端口，在同步时钟信号和 GR1 写入控制信号的作用下，GR1 内部寄存器更新存储内容，

将原数据替换为加法运算结果，最终完成 “GR1 = GR1 + GR2” 的功能，运算结果存储于 GR1，实现运算结果的留存与复用。

实现 GR1+GR2→GR1，将减法的结果存入寄存器 1 中的功能如图 4.5 所示

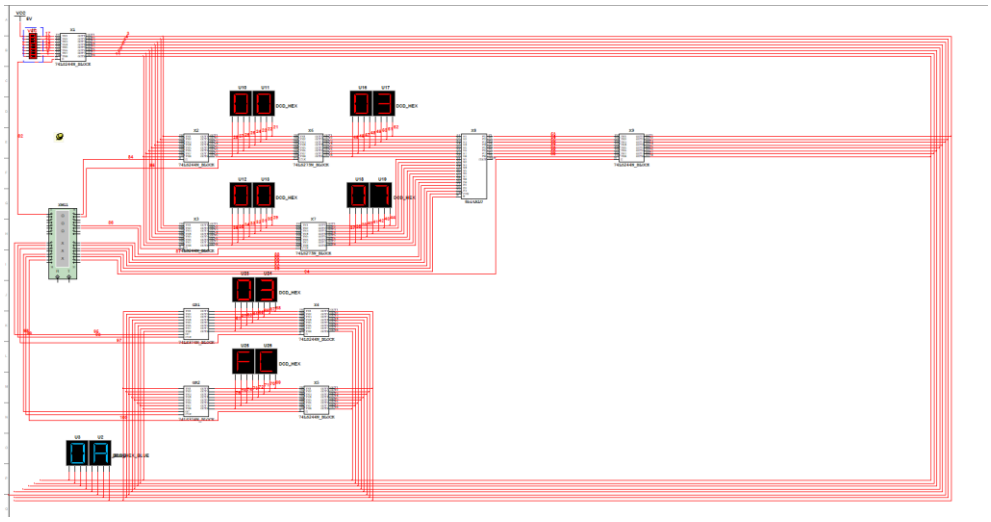


图 4.5 实现 GR1+GR2→GR1，将减法的结果存入寄存器 1 中

六、小结与心得体会

本次模拟微程序实现指令实验是对计算机组成原理中 “微程序控制” 核心概念的深度实践，通过字发生器模拟微指令、驱动单总线运算器自动执行指令，让我对硬件与控制逻辑的协同关系有了更直观的认识。